

American Sign Language Fingerspelling Recognition

A Comparative Study of Conventional Classifiers and Neural Networks

Corentin Le Bris, Toni Ortiz, Yeray Espinosa

Group Iota

github.com/yeraay21/ASL-fingerspelling

1. Introduction

Fingerspelling is the part of American Sign Language (ASL) where words are spelled out one letter at a time using static hand shapes. It is used for names, technical terms, and any word that has no dedicated sign. A system that can read these hand shapes from a camera would help close part of the communication gap between signers and people who do not know ASL, and it can serve as a building block for larger sign-recognition tools.

This project treats fingerspelling recognition as an image classification problem and asks a practical question: on the same data, how much does the choice of method actually matter? We compare three approaches that represent different moments in computer vision. The first uses handcrafted Gabor features with a Support Vector Machine and learns no features at all. The second is a convolutional neural network (CNN) trained from scratch. The third reuses a MobileNetV2 network already trained on ImageNet and adapts it to our task through transfer learning.

We evaluate all three under a cross-subject protocol, where the person shown at test time never appears during training. This is harder than a random split because the model cannot memorize a specific hand or background, and it gives a more honest estimate of how the system would behave with a new user.

2. State of the Art

Early work on hand gesture recognition relied on handcrafted features. Descriptors such as Gabor filters, HOG, or SIFT were computed from the image and passed to classifiers like SVMs. Gabor filters in particular respond well to oriented textures, which suits the ridges of a hand. These pipelines were cheap to train, but their accuracy depended heavily on feature engineering and degraded under changes in lighting or viewpoint.

Convolutional neural networks (CNNs) changed this by learning features directly from pixels. A network trained end-to-end usually beats a handcrafted pipeline given enough data. However, training a deep CNN from scratch requires a massive and varied dataset, and it tends to overfit when data is limited or when the same people appear in both the training and testing sets.

Transfer learning is the usual answer to that problem. A network pre-trained on a large dataset like ImageNet already contains general edge and shape detectors, so only the later layers need to be adapted. MobileNetV2 is a common choice because it remains lightweight while keeping good accuracy. Progressive unfreezing—training the head first and releasing deeper layers gradually—is a standard recipe for fine-tuning without destroying the pre-trained weights.

Much of the published work on ASL recognition reports results on random train and test splits, which can hide how strongly a model memorizes subject-specific cues. This project keeps the three

paradigms above and evaluates them under a strict cross-subject split, ensuring the comparison reflects true generalization to an unseen person rather than memorization.

3. Methodology

3.1 Data Analysis

We use the Kaggle ASL Fingerspelling Images dataset, containing RGB and depth images from five subjects performing the 24 static letters (A–Y). Letters J and Z are excluded as they require motion. Only RGB images are used in this work.

To test true generalization, we employ a strict cross-subject split. Subjects 1 to 4 form the training pool (77,549 images), from which 10% is set aside for validation using a stratified split. Subject 5 is held out entirely as the test set (24,000 images). Because this subject is never seen during training, the models cannot rely on memorizing specific hand shapes, skin tones, or backgrounds, ensuring the test results reflect genuine cross-subject performance.

Preprocessing varies by model. The Gabor pipeline uses 64x64 grayscale images scaled from 0 to 1. The custom CNN uses 64x64 RGB images (0 to 1 scaling, without ImageNet normalization). Conversely, MobileNetV2 requires its pre-training resolution, so inputs are resized to 224x224 RGB and normalized using ImageNet statistics (mean [0.485, 0.456, 0.406], std [0.229, 0.224, 0.225]). Data augmentation is applied exclusively during the custom CNN training and is detailed in its respective section.



Figure 1: Sample RGB images from the ASL Fingerspelling dataset

3.2 Models and Optimization

3.2.1 Gabor Filters + SVM

The first model is a classical computer vision pipeline based on handcrafted Gabor features and an SVM classifier. Unlike the CNN-based methods, this model does not learn visual filters from the images. The feature extraction step is fixed in advance and designed to capture low-level structures such as edges, orientations, textures, and local frequency changes.

Each RGB image is converted to grayscale and resized to 64x64. We then apply a bank of 40 Gabor filters, built from 8 orientations and 5 frequencies. Each filter is applied by convolution and produces a response map showing where a specific orientation-frequency pattern appears in the image.

To obtain a compact representation, each response map is summarized by two values: its mean and standard deviation. The mean describes the average response of the filter, while the standard

deviation measures how much this response varies across the image. With 40 filters, this gives 80 features per image.

The features are standardized using the training set statistics, then PCA reduces the vector from 80 to 60 components. This removes part of the redundancy between correlated Gabor features while keeping a relatively rich representation. Finally, the reduced vectors are classified with an SVM using an RBF kernel, which allows non-linear decision boundaries in the Gabor-PCA feature space. The penalty parameter C is selected by grid search with cross-validation, and the best value is $C = 10$.

3.2.2 Custom CNN

The second model is a Convolutional Neural Network (CNN) built entirely from scratch to classify the 64x64 RGB images without relying on pre-trained weights. The architecture consists of two main parts: a feature extractor and a classifier. The feature extractor is composed of three convolutional blocks with 32, 64, and 128 channels, respectively. Each block applies a 3x3 convolution with a padding of 1, followed by Batch Normalization, a ReLU activation, and 2x2 Max Pooling to progressively reduce spatial dimensions while increasing the channel depth. The classifier flattens the resulting feature maps into a 1D vector and passes them through a fully connected layer of 256 neurons with ReLU activation, culminating in a final linear layer with 24 outputs.

Because the model is trained from scratch on a relatively limited dataset, it was highly susceptible to severe overfitting, initially memorizing the training subjects and failing to generalize to the unseen Subject 5. To counteract this, an aggressive regularization strategy was implemented. First, Spatial Dropout (Dropout2d) was added at the end of each convolutional block with progressively increasing rates (0.1, 0.15, and 0.2), along with a standard Dropout of 0.5 in the dense layer. Second, strong data augmentations were applied exclusively during training, including ColorJitter, RandomPerspective, GaussianBlur, and RandomErasing, forcing the network to learn robust geometrical features rather than relying on skin color or lighting. Finally, the Cross-Entropy loss function was modified with a Label Smoothing factor of 0.1 to prevent the model from becoming overconfident in its predictions. The network was optimized using Adam with a learning rate of 5×10^{-4} and a batch size of 64 over 40 epochs.

3.2.3 MobileNetV2 (Transfer Learning)

The transfer learning model is built on MobileNetV2 with weights pre-trained on ImageNet. The original 1000-class classifier is removed and replaced by a new linear layer with 24 outputs, one per letter. The convolutional backbone is reused as a feature extractor and adapted to the ASL images through fine-tuning. All inputs are resized to 224x224 and normalized with the ImageNet statistics so they match what the backbone was trained on.

Fine-tuning follows a progressive unfreezing schedule with three phases, all optimized with Adam and cross-entropy loss. In the first phase the backbone is fully frozen and only the new head is trained for 3 epochs at a learning rate of $1e-3$, which lets the head adapt to the pre-trained features without disturbing them. In the second phase the backbone layers from index 14 onwards are unfrozen and trained with the head for 5 epochs at $1e-4$, a smaller rate so the pre-trained weights move only slightly. In the third phase the entire network is unfrozen and fine-tuned end to end for 5 epochs at $1e-5$. The batch size is 32, reduced from 64 because the 224x224 inputs use more GPU memory.

Hyperparameter	Value
Backbone	MobileNetV2 (ImageNet pre-trained)
Output classes	24 (letters A-Y, excluding J and Z)
Optimizer	Adam (all phases)
Loss function	Cross-entropy
Input resolution	224x224 RGB
Input normalization	ImageNet mean [0.485, 0.456, 0.406], std [0.229, 0.224, 0.225]
Batch size	32 (reduced from 64 due to VRAM at 224x224)
Phase 1: head only	3 epochs, lr 1e-3, backbone frozen
Phase 2: partial unfreeze	5 epochs, lr 1e-4, layers from index 14 onward unfrozen
Phase 3: full unfreeze	5 epochs, lr 1e-5, entire network unfrozen

Table 1: Hyperparameters and progressive unfreezing schedule for the MobileNetV2 model.

4. Experiments

All three methods are trained on the same subjects (1 to 4) and evaluated on the same held-out subject (5), so any difference in the final numbers comes from the method and not from the data split. The main experiment is the comparison of the three paradigms under this cross-subject protocol. For each model we report global accuracy, macro precision, macro recall, and macro F1 on subject 5, together with a confusion matrix to see which letters are mixed up.

4.1 Gabor + SVM

The Gabor + SVM experiment serves as the handcrafted baseline of the project. Its purpose is to measure how well a classical feature extraction method can perform under the same cross-subject protocol as the neural-network models.

The model is trained after the full Gabor preprocessing pipeline described above: grayscale conversion, Gabor feature extraction, standardization, PCA reduction, and RBF-SVM classification. The value of C is selected by cross-validation on the training data only.

On subject 5, we report accuracy, macro precision, macro recall, and macro F1-score. We also compute a Precision-Recall curve from the SVM decision scores, and we analyze normalized zoom-ins of the confusion matrix to identify the most recurrent error patterns.

4.2 Custom CNN experiments

The custom CNN experiments aimed to evaluate the network's generalization to an unseen subject (Subject 5) when trained from scratch, and to assess the effectiveness of our aggressive

regularization. The model was trained under the standard cross-subject protocol (Subjects 1–4 for training/validation, Subject 5 for testing).

We monitored convergence and overfitting through loss and accuracy curves over 40 epochs. On the test set, we report global accuracy alongside macro precision, recall, and F1-score. Furthermore, we generated a full confusion matrix to identify class-level ambiguities. Since the network struggled heavily with closed-fist signs, we isolated the top misclassifications for the most problematic category (Class A) to understand failure modes at a 64x64 resolution.

4.3 MobileNetV2: progressive unfreezing

For the MobileNetV2 model the specific experiment is the progressive unfreezing schedule. We train in three stages and record the training and validation curves at each stage, so the effect of every round of unfreezing can be measured. The first stage trains only the head, the second unfreezes the deeper blocks, and the third fine-tunes the full network.

5. Results

5.1.1 Gabor + SVM results

On subject 5, we got a global accuracy of 23%, a macro precision of 0.24, a macro recall of 0.23 and a macro F1 of 0.21. We also display the Precision-Recall curve right below.

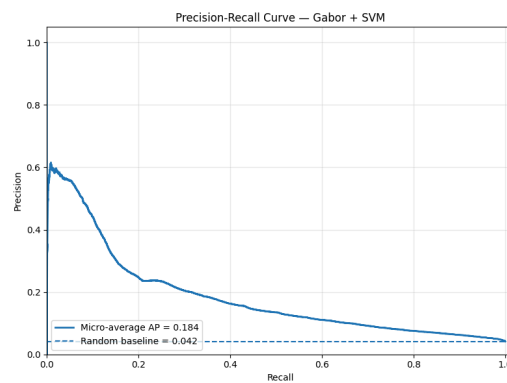


Figure 2: Precision-Recall curve for the Gabor + SVM baseline model evaluated on Subject 5.

Here are a few zoom-ins of the confusion matrix, showing some recurrent mistakes:

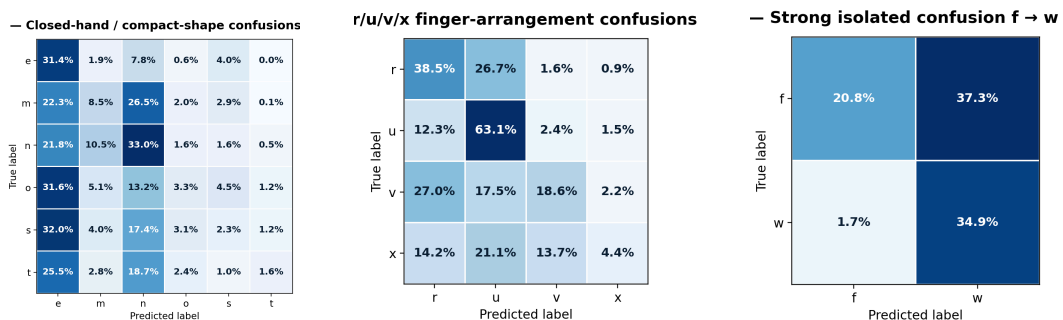


Figure 3: Confusion matrix zoom-ins highlighting the most recurrent error patterns (Gabor + SVM)

5.1.2 Custom CNN results

On the held-out Subject 5, the regularized custom CNN achieved a global test accuracy of 55.0%. The macro-averaged metrics reflect a model that is relatively precise but struggles to retrieve all instances, yielding a macro precision of 0.71, a macro recall of 0.55, and a macro F1-score of 0.57.

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
Custom CNN	55%	0.71	0.55	0.57

Table 2: Test performance metrics for the Custom CNN model evaluated on the held-out Subject 5.

The training curves demonstrated stable and smooth convergence over the 40 epochs. Notably, the validation loss consistently remained lower than the training loss; this expected behavior is a direct consequence of the aggressive regularization (Spatial Dropout and heavy data augmentations) which penalizes the training phase but is deactivated during validation.

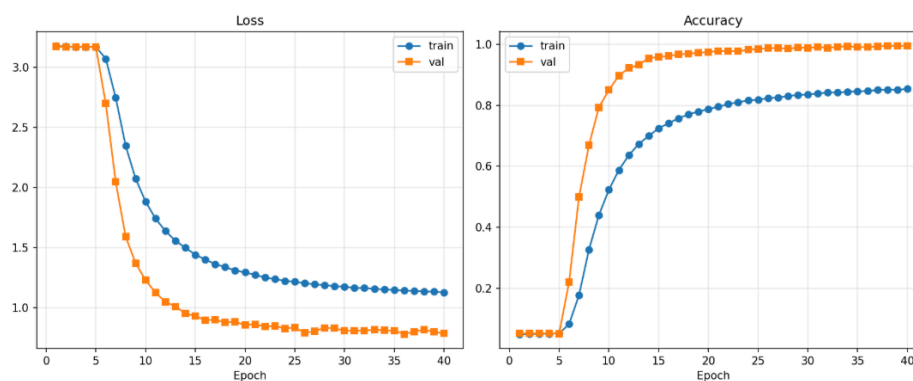


Figure 4: Training and validation loss and accuracy curves for the Custom CNN over 40 epochs.

The model showed strong performance on distinct, open-hand signs, achieving the highest recall rates on classes H (98.7%), C (97.1%), A (89.2%), and P (84.1%). However, an analysis of the confusion matrix revealed a critical bottleneck: the model struggles severely with closed-fist letters (T, M, S, E, N). A dedicated false-positive analysis showed that class A acts as a massive "sink", absorbing hundreds of misclassifications from these visually similar closed-fist signs.

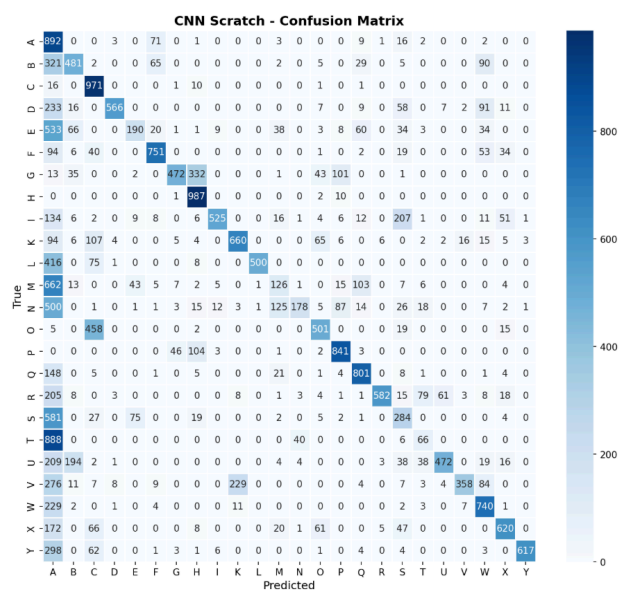
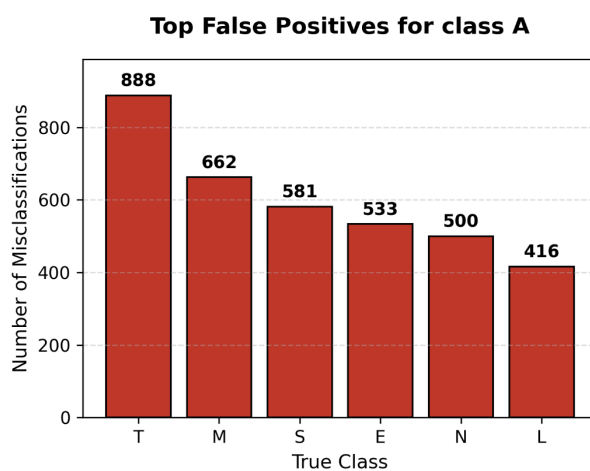


Figure 5: Most frequent false positives for class A.

Figure 6: Custom CNN confusion matrix (subj 5)

5.1.3 MobileNetV2 results

On the held-out subject 5, MobileNetV2 reaches a global accuracy of 81.52%, with a macro precision of 0.86, a macro recall of 0.82, and a macro F1 of 0.81. During training the loss falls quickly and the accuracy on the training data approaches 100%.

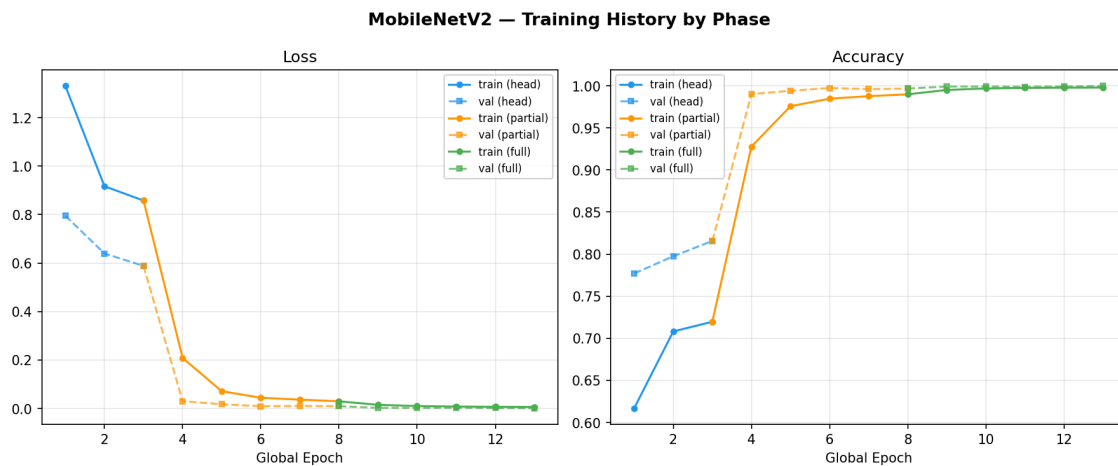


Figure 7: MobileNetV2 training and validation history across the three unfreezing phases.

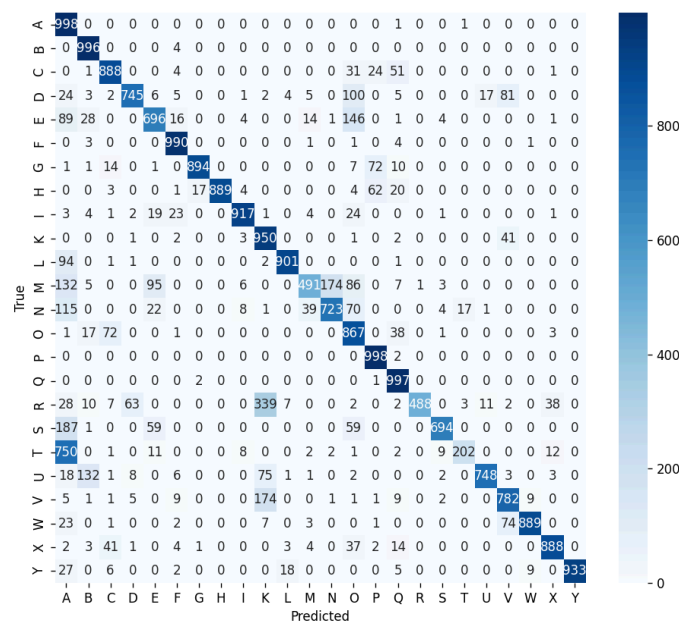


Figure 8: Full confusion matrix for the MobileNetV2 model evaluated on the held-out Subject 5.

The most frequent letter confusions on subject 5 are listed below, counted from the confusion matrix.

True	Predicted	Errors
T	A	750
R	K	339
S	A	187

V	K	174
M	N	174
E	O	146
U	B	132
M	A	132
N	A	115
D	O	100

Table 3: Top 10 most frequent letter misclassifications produced by the MobileNetV2 model.

5.2 Model comparison

The table below compares the three methods on subject 5 under the same protocol.

Model	Accuracy	Macro Prec.	Macro Recall	Macro F1
Gabor + SVM	23%	0.24	0.23	0.21
Custom CNN	55%	0.71	0.55	0.57
MobileNetV2	82%	0.86	0.82	0.81

Table 4: Performance comparison of the 3 evaluated models under the cross-subject testing protocol.

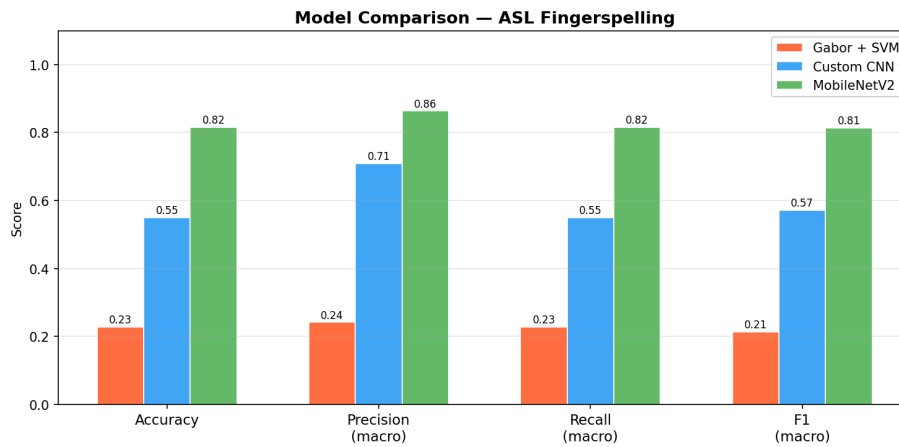


Figure 9: Visual comparison of the final performance metrics for the three evaluated models.

6. Discussion

6.1.1 Gabor + SVM

The Gabor + SVM model gives the weakest performance of the three approaches, with about 23% test accuracy, 24% macro precision, 23% macro recall, and 21% macro F1-score. These results are above random guessing for 24 classes, but they remain limited. The model extracts some useful visual information, but the handcrafted representation is not discriminative enough to separate all ASL letters reliably.

The Precision-Recall curve confirms this. The micro-average AP is 0.184, compared with a random baseline of 0.042. The model is therefore not random, but precision drops quickly when recall increases. In practice, it can make some confident correct predictions, but it produces many false positives when trying to recover more true samples.

The confusion matrix shows that the errors are mainly concentrated between visually similar signs. Compact or closed-hand signs are often predicted as e or n, while v and x are often predicted as r or u. A strong isolated confusion also appears between f and w, with many true f samples predicted as w. These patterns suggest that the classifier captures some global hand structure, but misses fine spatial details such as finger placement, thumb position, or small differences in hand configuration.

This limitation is consistent with the Gabor representation. Gabor filters detect low-level visual patterns such as orientations, edges, textures, and local frequencies. However, in this pipeline each response map is reduced to only its mean and standard deviation, which removes much of the spatial information. Two signs may therefore have similar global Gabor statistics even if their fingers are arranged differently.

Overall, Gabor + SVM is useful as a simple and interpretable baseline. It shows that fixed low-level filters can capture part of the hand structure, but they are not flexible enough for the full recognition task. This motivates the deep learning approaches, which learn task-specific features directly from the data and preserve spatial representations through several layers.

6.1.2 Custom CNN

The custom CNN demonstrates a significant leap in performance over the handcrafted Gabor baseline, proving the superiority of learning features directly from pixels. The 55.0% accuracy on an unseen subject represents a massive absolute improvement (over 33%) compared to the initial unregularized iterations of the same architecture, which suffered from severe subject-level memorization. This validates the necessity of heavy augmentations, Spatial Dropout, and Label Smoothing when training deep networks from scratch on limited cross-subject datasets.

Despite the improvements, the macro F1-score of 0.57 highlights a clear limitation in recall. The model's primary failure mode—misclassifying closed-fist letters (T, M, S, E, N) as A—exposes the limitations of the 64x64 input resolution. In low-resolution images, the subtle spatial differences in thumb placement that distinguish these signs are easily lost, causing the network to group them into the most dominant geometrical shape (Class A).

While the custom CNN successfully mitigates overfitting and learns generalized representations of hands, it lacks the highly refined, general-purpose edge and texture detectors present in models pre-trained on millions of images. This lack of initial visual intuition is what ultimately prevents it from reaching the high-accuracy tier achieved by the transfer learning approach, leaving it as a solid, but limited, intermediate step between traditional computer vision and large-scale deep learning.

6.1.3 MobileNetV2

The MobileNetV2 result matches what transfer learning usually delivers. The backbone already knows general visual features from ImageNet, so the model does not have to learn edges and textures from the ASL data and can spend its capacity on telling the letters apart. This is the main reason it reaches 81.52% on a subject it never saw, well above the other two methods.

The gap between validation and test is the clearest signal in these numbers. Validation accuracy on subjects 1 to 4 sits at 99.99% while the cross-subject test is 81.52%, so a good part of what the network learns is specific to the training subjects rather than to the letters themselves. The model is still the most robust of the three, but this gap is exactly the subject-level overfitting that the cross-subject split was meant to expose.

The remaining errors line up with how the hand shapes look. The dominant confusion is T predicted as A, with 750 cases, which is expected because both letters curl the fingers over the thumb and differ mainly in thumb placement. The M and N pair is the classic ASL ambiguity, where the only difference is one extra finger folded over the thumb. R and V are often read as K, since all three use extended fingers in similar configurations. Several letters also collapse onto A, which behaves as a frequent sink for closed, curled-hand shapes.

6.2 Cross-model comparison

Read together, the three methods trace the same story seen in the wider literature. The handcrafted Gabor pipeline carries no learned representation and sits lowest. The CNN trained from scratch learns useful features but is limited by the amount of data and by cross-subject variation. The pre-trained MobileNetV2 starts from strong general features and ends clearly ahead. The same data and the same held-out subject sit behind all three numbers, so the ranking comes from the method used.

6.3 Limitations

All models are trained and tested on a single dataset, so the numbers may not transfer to other cameras or settings. The two from-scratch pipelines run at 64x64, which already loses fine detail such as thumb position. Performance is sensitive to lighting and to complex backgrounds, and a single held-out subject gives only one estimate of cross-subject performance.

7. Conclusions

- Transfer learning with MobileNetV2 was the strongest method, reaching 81.52% accuracy on a subject never seen during training.
- The cross-subject split made the task harder and gave a more realistic measure of generalization than a random split would. The 99.99% validation versus 81.52% test gap shows clear subject-level overfitting.
- The CNN trained from scratch and the classical Gabor + SVM pipeline both trailed the pre-trained model, confirming the value of pre-trained features when data per subject is limited.
- The hardest cases came from visually similar hand shapes, such as T against A and the M/N pair, and from images with difficult lighting or backgrounds.
- As future work, Leave-One-Subject-Out cross-validation would give a more reliable performance estimate; using the depth channel and trying lighter input resolutions are natural extensions.

8. References

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). "ImageNet: A large-scale hierarchical image database". In 2009 IEEE Conference on Computer Vision and Pattern Recognition (pp. 248-255).

Kingma, D. P., & Ba, J. (2015). "Adam: A Method for Stochastic Optimization". In 3rd International Conference on Learning Representations (ICLR).

Paszke, A., et al. (2019). "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In Advances in Neural Information Processing Systems 32 (pp. 8024-8035).

Pedregosa, F., et al. (2011). "Scikit-learn: Machine Learning in Python". Journal of Machine Learning Research, 12, 2825-2830.

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). "MobileNetV2: Inverted Residuals and Linear Bottlenecks". In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 4510-4520).

Shoaib, M. (n.d.). "American Sign Language" [Dataset]. Kaggle. Available:
<https://www.kaggle.com/datasets/shoaib98libra/american-sign-language>

Shoaib, M. (n.d.). "ASL-Fingerspelling" [Source code]. GitHub. Available:
<https://github.com/Mohammed-Shoaib/ASL-Fingerspelling>